EX:5 WRITE THE PROGRAM FOR MISSIONARIES AND CANNIBALS PROBLEM

AIM

To develop a Python program that solves the **Missionaries and Cannibals Problem** by safely transporting all missionaries and cannibals across the river without leaving missionaries outnumbered by cannibals on either riverbank.

ALGORITHM

Step 1: Start the program.

Step 2: Define the initial state (3 Missionaries, 3 Cannibals, Boat on the left bank) and the goal state (all on the right bank).

Step 3: Generate all possible valid boat moves (one or two passengers).

Step 4: Check each new state to ensure missionaries are never outnumbered by cannibals on either bank.

Step 5: Continue exploring valid states until the goal state is reached.

Step 6: Display the sequence of safe moves from the initial state to the goal state.

Step 7: End the program.

PROGRAM CODE:

```
from collections import deque


M = int(input("Enter number of Missionaries: "))

C = int(input("Enter number of Cannibals: "))


def valid(m, c):
    if m < 0 or c < 0 or m > M or c > C:
        return False
    if m > 0 and m < c:
        return False
    if (M - m) > 0 and (M - m) < (C - c):
```

```
        return False
    return True

moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]

start = (M, C, 1)
goal = (0, 0, 0)

queue = deque([(start, [])])
visited = set()

while queue:
    state, path = queue.popleft()

    if state in visited:
        continue
    visited.add(state)

    if state == goal:
        print("\nSolution Path:")
        for s in path + [state]:
            m, c, boat = s
            side = "Left" if boat == 1 else "Right"
            print(f"({m}, {c}, {side})")
        break

    m, c, boat = state
```

```
    for dm, dc in moves:
        if boat == 1:
            new = (m - dm, c - dc, 0)
        else:
            new = (m + dm, c + dc, 1)

        if valid(new[0], new[1]):
            queue.append((new, path + [state]))
```

OUTPUT:

```
Enter number of Missionaries: 3
Enter number of Cannibals: 3

Solution Path:
(3, 3, Left)
(3, 1, Right)
(3, 2, Left)
(3, 0, Right)
(3, 1, Left)
(1, 1, Right)
(2, 2, Left)
(0, 2, Right)
(0, 3, Left)
(0, 1, Right)
(1, 1, Left)
(0, 0, Right)
>
```

RESULT:

The Python program successfully finds a safe sequence of moves to transport all missionaries and cannibals across the river. The solution path from the initial state to the goal state is displayed successfully.